PYTHON FOUNDATIONS - ASSIGNMENT 06

Loops

for, while, sum & count patterns

26 progressive Python challenges: basic for loops, range(), accumulators, counters, mixed practice and while loops.

for | range() | sum | count | while | if / else

Name: _____

Date: _____

Batch: _____

BEGINNER TRACK

26 PROGRESSIVE CHALLENGES

Quick reference before you begin

Build the loop logic before writing the full program.

Trace the changing variable on paper when a sum, count or while condition feels confusing.

for loop

for i in range(...)

repeat a known number of times

range()

start, stop, step

stop value is excluded

while loop

repeat while a condition is True

update the condition variable

1 Accumulator / SUM

Start total = 0, then add each required value to total.

2 Counter / COUNT

Start count = 0, then increase it only when the condition matches.

3 Loop + condition

Use if / elif / else inside a loop to classify each iteration.

4 Avoid infinite while loops

Make sure something inside the loop can eventually make the condition False.

Important: Use only concepts already covered: input, type conversion, arithmetic, comparison/logical operators, membership, if/elif/else and loops. No functions required.

Basic for loop + range()

Start with repetition, direction and step values.

1 Number Walk: 1 to 20

FOR RANGE

Warm up with the simplest counted loop.

1. Use a **for** loop to print every number from 1 to 20, one number per line.
2. Do not write twenty print statements.
3. After the loop, print **Loop completed**.

2 Reverse Countdown

FOR RANGE STEP

Practice a negative step and understand where range() stops.

1. Print numbers from 15 down to 1 using a for loop.
2. Use the third argument of range() instead of doing subtraction inside print().
3. After 1, print **Go!**.

3 Even Number Generator

FOR RANGE STEP

Generate values directly with range() rather than checking every number.

1. Print all even numbers from 2 to 40.
2. Use a step value so only even numbers are generated.
3. Print the text **Even:** before each number.

Basic for loop + range()

Start with repetition, direction and step values.

4 Odd Number Generator

FOR RANGE STEP

Use start, stop and step together.

1. Print all odd numbers from 1 to 39.
2. Do not use an if statement for this question.
3. Make sure 39 is included.

5 Multiples of Five

FOR RANGE

Build confidence with a predictable numerical sequence.

1. Print 5, 10, 15 ... up to 100.
2. Use range() so the loop itself generates the multiples.
3. Print each value as **Multiple of 5: value**.

6 User-Defined Range

INPUT FOR RANGE

Let the user decide the boundaries of the loop.

1. Take a starting number and ending number from the user.
2. Print every integer from start to end, including the ending number.
3. Assume the starting value is smaller than the ending value.

Accumulator and counter patterns

Learn how loops remember totals and counts across iterations.

7 Sum from 1 to N

FOR SUM INPUT

Introduce the accumulator pattern: a variable keeps the running total.

1. Take N from the user.
2. Start total at 0.
3. Use a for loop to calculate 1 + 2 + ... + N.
4. Print the final sum only after the loop ends.

8 Sum of Even Numbers

FOR SUM %

Combine arithmetic and a loop to total only selected values.

1. Calculate the sum of all even numbers from 1 to 100.
2. You may generate even values directly with range() OR check each number using %.
3. Print the final sum.

9 Multiples-of-Three Total

FOR SUM CONDITION

Accumulate values only when a condition matches.

1. Loop from 1 to 60.
2. Add a number to total only when it is divisible by 3.
3. Print the total at the end.
4. Also print how many numbers were added.

Accumulator and counter patterns

Learn how loops remember totals and counts across iterations.

10 Even vs Odd Counter

FOR COUNT %

Use two counters inside the same loop.

1. Take N from the user.
2. Loop from 1 to N.
3. Count how many values are even and how many are odd.
4. Print both counts after the loop.

11 Divisibility Counter Challenge

FOR COUNT LOGICAL

Count values that satisfy one condition and a more specific second condition.

1. Loop from 1 to 100.
2. Count how many numbers are divisible by 5.
3. Separately count how many are divisible by 5 **but not** divisible by 10.
4. Print both counts.

12 Five-Student Marks Analyzer

FOR SUM COUNT

Collect repeated user input without using a list.

1. Run the loop exactly 5 times.
2. Inside the loop, take one student mark from the user.
3. Maintain total marks, pass count (marks >= 40) and fail count.
4. After the loop, calculate and print the average, pass count and fail count.
5. Round the average to 2 decimal places.

Combine loops with earlier Python concepts

Use arithmetic, conditions, membership and repeated input.

13 Multiplication Table

FOR ARITHMETIC

Generate a complete table from one user input.

1. Take a number from the user.
2. Print its multiplication table from 1 to 10.
3. Format each line like **7 x 3 = 21**.

14 Three / Five / ThreeFive

FOR IF-ELIF %

Practice condition ordering inside a loop.

1. Loop from 1 to 50.
2. If a number is divisible by both 3 and 5, print **ThreeFive**.
3. Else if divisible by only 3, print **Three**.
4. Else if divisible by only 5, print **Five**.
5. Otherwise print the number itself.

15 Vowel Explorer

FOR MEMBERSHIP COUNT

Iterate through a string and reuse the membership operator.

1. Take one word from the user and convert it to lowercase.
2. Loop through every character.
3. If the character is in **aeiou**, print the vowel and increase a vowel counter.
4. Print the total number of vowels after the loop.

Combine loops with earlier Python concepts

Use arithmetic, conditions, membership and repeated input.

16 Five-Product Shopping Summary

FOR SUM COUNT

Use repeated input to produce a small shopping report without lists.

1. Run a loop 5 times and take one product price each time.
2. Calculate the total cost of all 5 products.
3. Count how many products cost more than 1000.
4. Calculate the average product price.
5. Print total, average and expensive-product count.

17 Student Eligibility Batch

FOR AND COUNT

Apply the same eligibility rule to several students.

1. Process exactly 5 students.
2. For each student, take marks and attendance percentage.
3. A student is eligible when marks >= 40 **and** attendance >= 75.
4. Print Eligible / Not Eligible for each student.
5. At the end print total eligible and not-eligible counts.

Combine loops with earlier Python concepts

Use arithmetic, conditions, membership and repeated input.

18 Selective Range Challenge

FOR OR NOT SUM

Combine logical operators, divisibility checks, a counter and an accumulator.

1. Take N from the user and loop from 1 to N.
2. Select numbers divisible by 3 **or** 5, but exclude numbers divisible by both 3 and 5.
3. Print every selected number.
4. Count the selected numbers and calculate their sum.
5. Print the final count and sum.

Condition-controlled repetition

Build while-loop confidence with counters, retries and sentinel values.

19 while Warm-Up: 1 to 10

WHILE COUNTER

Rewrite a basic counted loop using while.

1. Start a variable at 1.
2. Use a while loop to print 1 through 10.
3. Increase the variable correctly so the loop ends.

20 Countdown with while

WHILE DECREMENT

Practice changing a loop variable in the opposite direction.

1. Start at 10.
2. Print 10 down to 1 using while.
3. Decrease the variable by 1 each iteration.
4. After the loop, print **Go!**.

21 Sum 1 to N Using while

WHILE SUM INPUT

Rebuild the accumulator pattern using a while condition.

1. Take N from the user.
2. Use while to calculate the sum from 1 to N.
3. Maintain both a counter variable and a total variable.
4. Print the final total.

Condition-controlled repetition

Build while-loop confidence with counters, retries and sentinel values.

22 Multiplication Table with while

WHILE ARITHMETIC

Repeat a fixed calculation without using for or range().

1. Take a number from the user.
2. Use while to print its table from 1 to 10.
3. Format the output like **6 x 4 = 24**.

23 Even Numbers with while

WHILE STEP

Use a larger update step rather than checking every value.

1. Print all even numbers from 2 to 50 using while.
2. Start at 2 and increase by 2 each iteration.
3. Do not use a for loop.

24 Password Until Correct

WHILE COMPARISON

Use a condition-controlled loop when the number of attempts is not known.

1. Set the correct password to **python123**.
2. Ask the user for a password before entering the loop.
3. While the password is incorrect, print a message and ask again.
4. When the correct password is entered, stop looping and print **Login Successful**.

Condition-controlled repetition

Build while-loop confidence with counters, retries and sentinel values.

25 ATM PIN - Maximum 3 Attempts

WHILE AND COUNT

Combine an attempt counter with a second while condition.

1. Set a correct PIN, for example **4321**.
2. Allow a maximum of 3 attempts.
3. Continue while attempts are below 3 **and** the PIN is incorrect.
4. After the loop, print **Access Granted** if correct; otherwise print **Card Blocked**.
5. Do this without using a for loop.

26 Shopping Total Until 0

WHILE SENTINEL SUM

Use a sentinel value to stop a loop when the user decides there are no more items.

1. Ask the user to enter a product price.
2. Keep accepting prices while the entered price is not 0.
3. Add every non-zero price to a running total and count the number of products.
4. When the user enters 0, stop the loop.
5. Print the number of products, total amount and average price.
6. Handle the case where the first entered value is 0 without dividing by zero.